

Frequency, Relative Frequency and Cumulative Frequency Report

Frequency: How many times a value appears in your data.

Relative Frequency: The proportion (or percentage) of how often a value appears compared to the total number of rows.

$$\text{Relative Frequency} = \frac{\text{Frequency}}{\text{Total Rows}}$$

Cumulative Frequency: Running total of percentages (always ends at 100%).

(Note: Used Google Colab(cloud mode) to execute the code. Colab → use `files.upload()` or `drive.mount()`).

Other mode to execute the code:

- Local (Jupyter, VS Code, PyCharm) → use file path directly.
- Databricks/Spark → use Spark's Excel reader.)

Step 1: Upload the file into Colab. Read the Excel file into pandas.

- Use `files.upload()` → temporary workspace.
- Or `drive.mount('/content/drive')` → permanent storage in Google Drive.

```
import pandas as pd
from google.colab import files
#Upload the file and read the file
#uploaded = files.upload()
#df = pd.read_excel("Placement.xlsx")

#*****
#Read the file from the colab or drive directly
dataset = pd.read_excel("/content/Placement.xlsx")
.....
```

Jupyter Notebook

```
import pandas as pd
dataset = pd.read_excel("Placement.xlsx") # file in same folder
#If the file is in another folder:
dataset = pd.read_excel("C:/Users/Anitha/Documents/Placement.xlsx")
```

Step 2: Count how many rows are in the dataset.

```
import pandas as pd
from google.colab import files
#Upload the file and read the file
#uploaded = files.upload()
#df = pd.read_excel("Placement.xlsx")

#####
#Read the file from the colab or drive directly
dataset = pd.read_excel("/content/Placement.xlsx")
#####
print("Total rows in the dataset:", dataset.shape[0])
print("Total Columns in the dataset:", dataset.shape[1])
```

```
Total rows in the dataset: 215
Total Columns in the dataset: 15
```

Step 3: Select the column 'ssc_p' for analysis.

```
import pandas as pd
from google.colab import files
#Upload the file and read the file
#uploaded = files.upload()
#df = pd.read_excel("Placement.xlsx")
#####
#Read the file from the colab or drive directly
dataset = pd.read_excel("/content/Placement.xlsx")
#####
print("Total rows in the dataset:", dataset.shape[0])
print("Total Columns in the dataset:", dataset.shape[1])
column = 'ssc_p'
```

Step 4: Read the Excel file into pandas DataFrame.

```
import pandas as pd
from google.colab import files
#Upload the file and read the file
#uploaded = files.upload()
#df = pd.read_excel("Placement.xlsx")
#####
#Read the file from the colab or drive directly
dataset = pd.read_excel("/content/Placement.xlsx")
#####
print("Total rows in the dataset:", dataset.shape[0])
print("Total Columns in the dataset:", dataset.shape[1])
column = 'ssc_p'
freqtable = pd.DataFrame(columns=["Unique_Values", "Frequency", "Relative_frequency", "Cumsum"])
print(freqtable)
```

Output:
Total rows in the dataset: 215
Total Columns in the dataset: 15
Empty DataFrame
Columns: [Unique_Values, Frequency, Relative_frequency, Cumsum]
Index: []

Step 5: Calculate

- Frequency (count of each unique value)
- Relative Frequency (Frequency ÷ Total rows)
- Cumulative Frequency (running total of relative frequencies).

```
import pandas as pd
from google.colab import files
#Upload the file and read the file
#uploaded = files.upload()
#df = pd.read_excel("Placement.xlsx")
#*****
#Read the file from the colab or drive directly
dataset = pd.read_excel("/content/Placement.xlsx")
#*****
print("Total rows in the dataset:", dataset.shape[0])
print("Total columns in the dataset:", dataset.shape[1])
column = 'ssc_p'
freqtable = pd.DataFrame(columns=["Unique_Values", "Frequency", "Relative_frequency", "Cumsum"])
print(freqtable)

freqtable["Unique_Values"] = dataset[column].value_counts().index
freqtable["Frequency"] = dataset[column].value_counts().values
freqtable["Relative_frequency"] = freqtable["Frequency"] / dataset.shape[0]
freqtable["Cumsum"] = freqtable["Relative_frequency"].cumsum()
```

Step 6: Sort the values in the column called Unique_Values

1. sort_values() - pandas function that **reorders the rows** of your DataFrame based on the values in one or more columns.
2. by="Unique_Values" - Sort the DataFrame using the values in the column called Unique_Values. The order of rows will depend on the values in that column

3. ascending=True - Sort in increasing order (smallest to largest, A → Z, or 1 → 100).
(If set ascending=False, it would sort in decreasing order (largest to smallest, Z → A, or 100 → 1).)

```
freqtable["Unique_Values"] = dataset[column].value_counts().index
freqtable["Frequency"] = dataset[column].value_counts().values
freqtable["Relative_frequency"] = freqtable["Frequency"] / dataset.shape[0]
freqtable["Cumsum"] = freqtable["Relative_frequency"].cumsum()
#*****
#Sorting the data based on the Unique_values
freqtable = freqtable.sort_values(by="Unique_Values", ascending=True)
```

Step 7: Print the output for the frequency, Relative Frequency and Cumulative Frequency.
Or Save/Download the results into xlsx spreadsheet.

```

#####
#Sorting the data based on the Unique_values
freqtable = freqtable.sort_values(by="Unique_Values", ascending=True)
print(freqtable)

```

Output:

Total rows in the dataset: 215

Total Columns in the dataset: 15

Empty DataFrame

Columns: [Unique_Values, Frequency, Relative_frequency, Cumsum]

Index: []

	Unique_Values	Frequency	Relative_frequency	Cumsum
39	40.89	1	0.004651	0.706977
99	41.00	1	0.004651	0.986047
101	43.00	1	0.004651	0.995349
77	44.00	1	0.004651	0.883721
66	45.00	1	0.004651	0.832558
..	...	...	...	...
36	85.80	1	0.004651	0.693023
50	86.50	1	0.004651	0.758140
22	87.00	3	0.013953	0.576744
92	88.00	1	0.004651	0.953488
76	89.40	1	0.004651	0.879070

[103 rows x 4 columns]

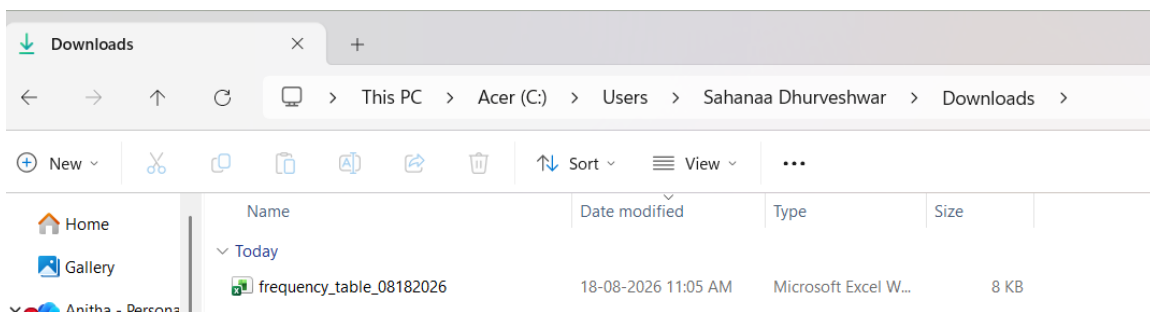
Step 7: Save/Download the results into xlsx spreadsheet

```

#####
#Sorting the data based on the Unique_values
freqtable = freqtable.sort_values(by="Unique_Values", ascending=True)
print(freqtable)
#####
#Download the output in Excel format
output_file = "frequency_table_NS.xlsx"
freqtable.to_excel(output_file, index=False)
files.download(output_file)

```

|



Conclusion:

- **Frequency** → Just counts how many times each value appears. Sorting doesn't change the counts; it only changes the order.
- **Relative Frequency** → $\text{Frequency} \div \text{Total rows}$. Since the counts don't change, these percentages also stay the same no matter how it gets sort.
- **Cumulative Frequency** → Running total of relative frequencies. This depends on the order. If you sort smallest to largest, the running total builds up differently than if you sort largest to smallest. But the final total always ends at **1.0 (100%)**, because added everything.

Code:

```
import pandas as pd
from google.colab import files
#Upload the file and read the file
#uploaded = files.upload()
#df = pd.read_excel("Placement.xlsx")
#*****
#Read the file from the colab or drive directly
dataset = pd.read_excel("/content/Placement.xlsx")
#*****
print("Output:")
print("Total rows in the dataset:", dataset.shape[0])
print("Total Columns in the dataset:", dataset.shape[1])
column = 'ssc_p'
freqtable = pd.DataFrame(columns=["Unique_Values", "Frequency",
"Relative_frequency", "Cumsum"])
print(freqtable)

freqtable["Unique_Values"] = dataset[column].value_counts().index
freqtable["Frequency"] = dataset[column].value_counts().values
freqtable["Relative_frequency"] = freqtable["Frequency"] /
dataset.shape[0]
freqtable["Cumsum"] = freqtable["Relative_frequency"].cumsum()
#*****
#Sorting the data based on the Unique_values
freqtable = freqtable.sort_values(by="Unique_Values",
ascending=True)
print(freqtable)
#*****
#Download the output in Excel format
output_file = "frequency_table_08182026.xlsx"
freqtable.to_excel(output_file, index=False)
```

```
files.download(output_file)
```